RuSMT

Speaker notes and rehearsal plan

Presenter: Meng Xu
University of Waterloo

Coauthor: Mehrad Haghshenas
SpecOps '26

38 main frames / 20:00 / no planned Q&A interval

11 additional overlay states are included in the frame timings. 10 backup frames are outside the timed talk. The live PDF contains 49 main presentation pages followed by the 10 backups. The review PDF contains one final state per logical frame.

The allocations are rehearsal targets, not measured speaking times. The suggested narration contains 2994 words; pointing, pauses, and overlay changes are additional parts of delivery. Keep the condition-specific question visible through the IMP example, the TOML exchange, and the results.

The story

Test a language implementation. Write a reference semantics and name selected conditions inside it. Derive the tests and oracle from that same source. IMP shows the authoring experience; TOML carries the empirical story.

Frames	Clock	Segment
1–6	00:00–02:30	Interpreter testing and the selected condition
7–15	02:30–07:50	Write and follow IMP; derive both uses
16–21	07:50–10:55	TOML branches and the solver limitation
22–27	10:55–14:05	Runner, prompt, candidate, feedback, and replay
28–31	14:05–15:55	Fixed-budget coverage, misses, and rejections
32–36	15:55–18:35	Conformance comparisons and triage
37–38	18:35–20:00	Boundaries and the central takeaway

Two distinctions to preserve

"Rejected by the oracle" is a document outcome. "Target reached" is the outcome of the testing request. A successful error-targeting test is supposed to be rejected by the oracle. Also, the paper's "errors as values" discipline is not a totality or panic-freedom guarantee.

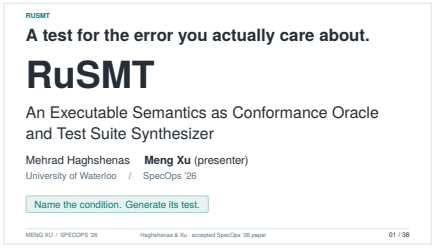
Source authority

The accepted paper is the basis for the technical claims, figures, and experimental results. Code-archive differences are not used to alter its claims. The code archive illustrates authoring and control flow. The Regorus origin is author-provided motivation, not an evaluated case study. All illustrations not copied as examples from the paper are marked as illustrations or schematics in the slides or notes. Paper DOI: 10.1145/3842652.3843196.

01 / A test for the error you actually care about.

00:00–00:15 · 15 seconds, including overlays

Review page 1; live PDF page(s) 1.



Haghshenas & Xu · accepted SpecOps ’26 paper

We want to test language implementations against the behaviors they are supposed to implement. RuSMT lets us ask for a test that reaches a particular condition in an executable specification, and uses the same source to provide the test oracle. Let me first make the testing setup concrete.

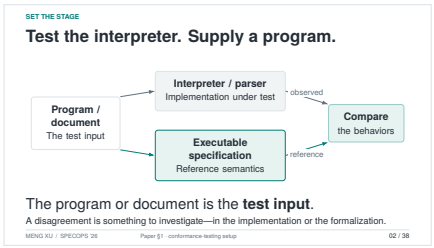
Delivery: Begin with the question, not a list of contributions. Full paper title and both authors are retained.

---

02 / Test the interpreter. Supply a program.

00:15–00:55 · 40 seconds, including overlays

Review page 2; live PDF page(s) 2-3.



Paper §1 · conformance-testing setup

The implementation under test is an interpreter or parser. The little program, or the configuration document, is the input we give it. To test conformance, we run the same input through that implementation and an executable specification, then compare the behaviors. The specification supplies the reference, but a disagreement can still mean the formalization is wrong. We need two things: a useful oracle and inputs that exercise the language behaviors we care about. An input-level error, such as division by zero, is not itself evidence that the interpreter is buggy. Handling it incorrectly could be.

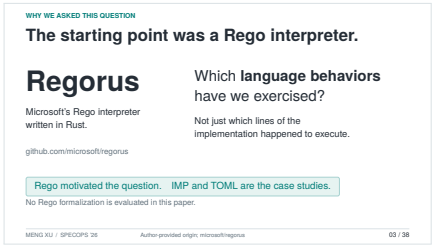
Delivery: First show the implementation and its input. Click to reveal the reference path and comparison. Point at the input box when distinguishing program from interpreter.

Precision: The diagram is the general testing setup. The paper does not evaluate independent IMP interpreters; the differential experiment is TOML accept/reject testing.

03 / The starting point was a Rego interpreter.

00:55–01:20 · 25 seconds, including overlays

Review page 3; live PDF page(s) 4.



This started from wanting to test Regorus, Microsoft's Rego interpreter. Fuzzing was one way to test the implementation, but we also wanted explicit coverage of Rego semantics, not only a report of which interpreter lines had run. That is the motivation for asking for selected behaviors in the language specification. We did not complete a Rego interpreter in RuSMT. Rego is the origin story, not a third evaluated case study; the paper demonstrates the approach with IMP and TOML.

Author-provided origin; microsoft/regorus

Delivery: Keep the origin story to this frame. Do not introduce Rego syntax or a Regorus bug example.

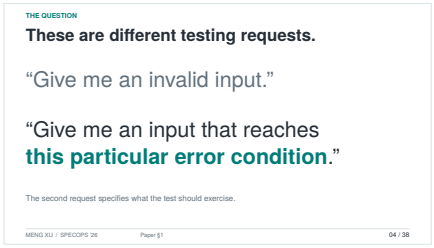
Precision: The research origin and incomplete Rego model are supplied by Meng Xu, not claimed as paper results. The repository supports only the project description.

---

04 / These are different testing requests.

01:20–01:40 · 20 seconds, including overlays

Review page 4; live PDF page(s) 5-6.



Once we have an oracle, where should its test inputs come from? These are two different requests: give me an invalid input, or give me an input that reaches this particular error condition. The second request tells us what behavior the test should exercise. RuSMT makes that target explicit inside the executable specification.

Paper §1

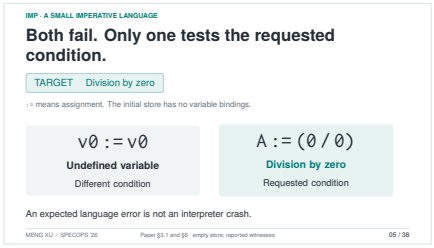
Delivery: Click once after reading the broad request. Pause on "this particular."

Precision: This contrasts measuring coverage with expressing an explicit target; it does not claim other testing techniques cannot target conditions.

05 / Both fail. Only one tests the requested condition.

01:40–02:15 · 35 seconds, including overlays

Review page 5; live PDF page(s) 7-8.



Paper §3.1 and §6 · empty store; reported witnesses

IMP is a small imperative language. The assignment notation is colon-equals, and we start with no variable bindings. Suppose the condition we want to exercise is division by zero. The program on the left reads an undefined variable, so it fails, but for the wrong reason. On the right, zero divided by zero reaches the condition we requested. Both programs fail. Only one tests what I asked for. These are the two reported witnesses placed side by side, not a history of a solver making a wrong guess.

Delivery: Show the undefined-variable program first. Click to reveal the division program. Keep the target chip visible.

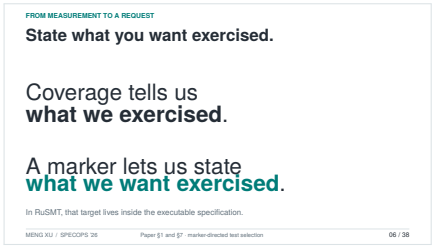
Precision: These are valid program forms with dynamic errors, not necessarily syntax errors. Two witnesses are juxtaposed for explanation.

---

06 / State what you want exercised.

02:15–02:30 · 15 seconds, including overlays

Review page 6; live PDF page(s) 9.



Paper §1 and §7 · marker-directed test selection

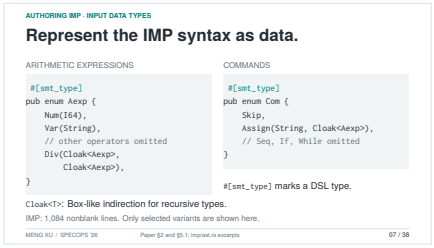
Coverage tells us what we exercised. A marker lets us state what we want exercised. Here the target will live in the reference semantics, rather than in the internals of the independent implementation. This is our interface choice, not a claim that other generators cannot target conditions. Now let me show what the author actually writes.

Delivery: Give the two sentences their own pause. Then move directly to code.

07 / Represent the IMP syntax as data.

02:30–03:10 · 40 seconds, including overlays

Review page 7; live PDF page(s) 10.



Paper §2 and §5.1; imp/ast.rs excerpts

The input program is an ordinary value of an annotated data type. Arithmetic expressions include numbers, variables, and division; commands include assignment. The omitted variants cover other arithmetic operators, sequencing, conditionals, and loops. The annotation marks a type for the RuSMT fragment. I64 is the library's 64-bit value type. Cloak is a Box-like wrapper so recursive syntax fits Rust's type rules. We are writing a reference interpreter here, not adding attributes to the production implementation. Two selected error markers do not mean that the whole IMP model has only two cases.

Delivery: Point to the two annotations, then Num / Var / Div and Assign. Do not read every constructor aloud.

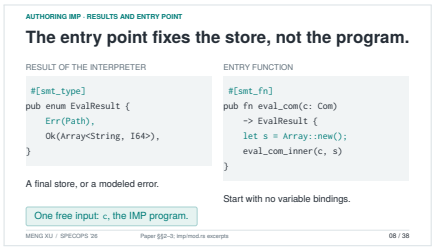
Precision: These are selected source variants; comments identify the elisions. The 1,084-line count is the paper's specification count, not the number of lines displayed.

---

08 / The entry point fixes the store, not the program.

03:10–03:55 · 45 seconds, including overlays

Review page 8; live PDF page(s) 11.



Paper §§2–3; imp/mod.rs excerpts

Here is the interface of the interpreter. Normal completion returns the final store, a map from names to values. A modeled error returns a Path. The entry function is annotated with smt-fn and takes a command, which is our input program. It creates the empty store itself and calls the recursive evaluator. That distinction matters later: in synthesis, RuSMT is allowed to choose the program, but it cannot invent an initial store that concrete execution would never use. The DSL library types and admitted operations have corresponding SMT meanings; this is restricted Rust, not arbitrary Rust.

Delivery: Explain the output type first. Then trace c and the newly created empty store into eval_com_inner.

Precision: Errors as values do not prove termination or that all library preconditions hold. Array is RuSMT's library map, not native Rust indexing.

09 / Assignment evaluates its expression first.

03:55–04:25 · 30 seconds, including overlays

Review page 9; live PDF page(s) 12.



IMP source · eval_com_inner (assignment arm)

This is the assignment arm of the command evaluator. It evaluates the right-hand expression first. If arithmetic evaluation returns an error, command evaluation returns the same path. If it returns a value, the evaluator produces an updated store. So the little division guard we are about to see is not a disconnected check. It lives in arithmetic evaluation, and its result propagates through the command evaluator to the entry point.

Delivery: Follow eval_aexp, then the two returned results. The reveal call just opens the recursive wrapper.

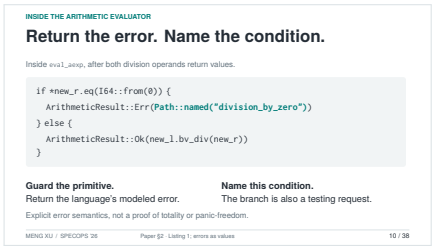
Precision: This is an excerpt of one match arm, not the entire command evaluator or a separately maintained test generator.

---

10 / Return the error. Name the condition.

04:25–05:15 · 50 seconds, including overlays

Review page 10; live PDF page(s) 13-14.



Paper §2 · Listing 1; errors as values

Now we are inside arithmetic evaluation, with values for both operands. The zero-divisor guard defines an error outcome instead of relying on the host operation to fail. We want the bad program to fail, not the specification running it. Otherwise, the evaluator calls the division primitive on its permitted inputs. Now look at Path named division-by-zero. That identifies this branch as a target for generation. The name is a handle; the surrounding code defines the condition. The author places the request where the behavior is defined. This is explicit error modeling, not a proof that every specification terminates or is panic-free; library preconditions still matter.

Delivery: First emphasize the guard and Err constructor. Click to emphasize Path::named without moving any code.

Precision: This fragment follows Listing 1 in the accepted paper. Do not claim totality, crash-freedom, or automatic verification of primitive preconditions.

11 / The other marker comes from reading the store.

05:15–05:45 · 30 seconds, including overlays

Review page 11; live PDF page(s) 15.



IMP source · store_get; paper §6 witness

The other marker is just as concrete. Store lookup checks whether the variable has a binding before selecting its value. Otherwise it returns the undefined-variable marker. In v-zero assigned to v-zero, the right-hand side reads that variable before any assignment has created it. The initial store is empty, so this is the branch reached. The two selected targets are attached to two different checks in the same reference semantics.

Delivery: Connect the contains_key test to the empty store established at the entry point.

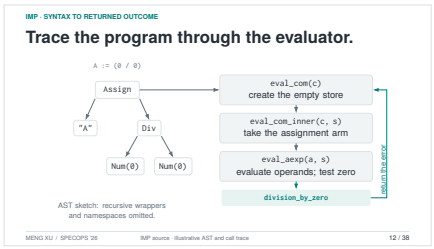
Precision: The excerpt retains the artifact's String::from conversion. This is illustrative source, not a new source of empirical claims.

---

12 / Trace the program through the evaluator.

05:45–06:15 · 30 seconds, including overlays

Review page 12; live PDF page(s) 16-17.



IMP source · illustrative AST and call trace

Here is how the pieces connect for A assigned to zero divided by zero. The source is represented by an assignment node containing a division expression and two numeric literals. The entry point creates the empty store. Command evaluation reaches the assignment arm, which calls arithmetic evaluation. The division guard returns the marker, and that error propagates back through the command to the entry result. This connected interpreter is what we compile and translate. The marker is not an isolated predicate pasted beside the implementation.

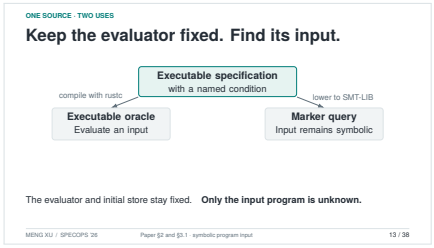
Delivery: Trace the syntax tree left, then follow the calls down the right. Click to reveal the returned marker and upward error propagation.

Precision: The AST is a schematic rendering: namespace prefixes, I64 constructors, and Cloak wrappers are omitted. It is not presented as compilable Rust.

13 / Keep the evaluator fixed. Find its input.

06:15–07:00 · 45 seconds, including overlays

Review page 13; live PDF page(s) 18-19.



Paper §2 and §3.1 · symbolic program input

Compile that source, and it evaluates a concrete program as our oracle. Translate the same annotated source, and RuSMT can ask for an input whose returned Path contains the selected marker. The interpreter and empty initial store stay fixed. Only the program is unknown. For IMP, that unknown is an abstract syntax tree. The correspondence is possible because the admitted types and operations have SMT encodings; we are not translating arbitrary Rust. There is no second hand-written generator describing how to manufacture a division-by-zero example.

Delivery: Show the execution route first, then reveal the query route. Keep the source box anchored.

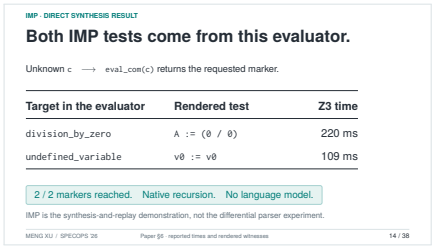
Precision: The annotations belong to the executable reference, not the independent implementation. RuSMT synthesizes the input to the interpreter, not a replacement interpreter.

---

14 / Both IMP tests come from this evaluator.

07:00–07:25 · 25 seconds, including overlays

Review page 14; live PDF page(s) 20.



Paper §6 · reported times and rendered witnesses

Now the solver result has a concrete source. Asking for the division marker gives the AST rendered as A assigned to zero divided by zero. Asking for the missing-variable marker gives v-zero assigned to v-zero. Z3 produces both models under native recursion, with the times shown. No language model is needed. IMP is our small end-to-end demonstration, not a differential comparison of production interpreters.

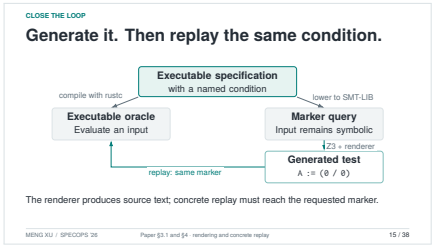
Delivery: Connect each target to its earlier guard. Do not linger on the millisecond difference.

Precision: The times and witness strings come from the accepted paper. Two selected markers are not a claim of complete semantic coverage.

15 / Generate it. Then replay the same condition.

07:25–07:50 · 25 seconds, including overlays

Review page 15; live PDF page(s) 21-22.



The model supplies an AST, which the renderer turns into source text. Concrete replay must reach the same marker in the compiled semantics. So the shared source supplies both the condition used to find this test and the executable oracle used to check its behavior. Replay checks agreement on the witness; it does not prove that the formalization faithfully captures the prose standard.

Paper §3.1 and §4 · rendering and concrete replay

Delivery: The generated test is present on arrival. Click to reveal its replay path to the oracle.

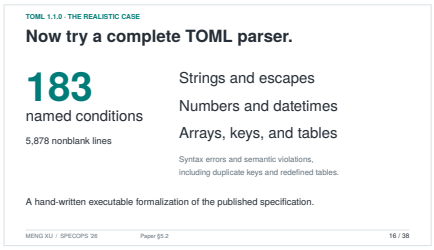
Precision: Use the paper's replay acceptance rule. Do not reintroduce archive discrepancies.

---

16 / Now try a complete TOML parser.

07:50–08:15 · 25 seconds, including overlays

Review page 16; live PDF page(s) 23.



Now move from the explanatory interpreter to a realistic parser. We wrote a TOML 1.1.0 specification with 183 named conditions across strings, numbers, datetimes, arrays, keys, and tables. It is about 5,900 nonblank lines. Here the symbolic input is the document itself, rather than an IMP syntax tree. The markers include both syntax errors and semantic violations.

Paper §5.2

Delivery: Give scale without reading an inventory. Remind the audience that the formalization is manually written.

17 / Two opening brackets. Two closing brackets.

08:15–08:30 · 15 seconds, including overlays

Review page 17; live PDF page(s) 24.

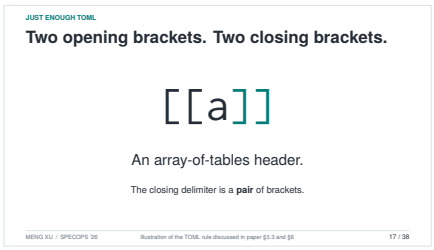


Illustration of the TOML rule discussed in paper §3.3 and §6

We need only one TOML construct for the running example. An array-of-tables header starts with two opening brackets and closes with two closing brackets. The key a names the table. We are interested in the reference parser's error behavior when the complete closing delimiter is missing. Let us look at the code that distinguishes those errors.

Delivery: Point to the closing pair, not to TOML's broader table semantics.

---

18 / First try the complete closing delimiter.

08:30–09:20 · 50 seconds, including overlays

Review page 18; live PDF page(s) 25.



TOML source · table.rs, reformatted excerpt

The opening brackets and key have already been parsed. The reference saves the state immediately after the key. It then asks a helper to parse the full closing delimiter: optional whitespace followed by two closing brackets. If that succeeds, parsing continues. NoMatch means that delimiter was not recognized; it is not yet one of the two named errors. The caller then inspects the character at the saved after-key position to classify the error. That saved position matters: we are not checking where the helper happened to stop after trying part of the delimiter. The next slide expands this exact NoMatch arm.

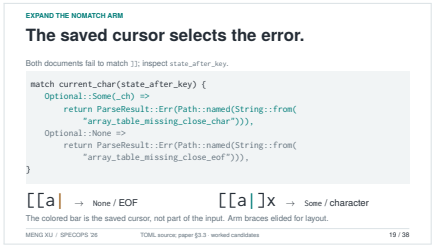
Delivery: Point at state_after_key in the helper call and again in current_char. The same saved state is passed to both.

Precision: The outer key/opening cases are omitted; the NoMatch arms are explicitly deferred. The excerpt is reformatted source, not a standalone function or a new implementation.

19 / The saved cursor selects the error.

09:20–10:05 · 45 seconds, including overlays

Review page 19; live PDF page(s) 26-27.



TOML source; paper §3.3 · worked candidates

Here are the two branches. If a character remains at the saved position, the parser returns the missing-close-character marker. If no character remains, it returns the missing-close-EOF marker. For double opening bracket a, the after-key position is already at end of input. For double opening bracket a closing bracket x, the complete double-bracket delimiter still fails, but a character is present at that saved position. So it reaches the character marker. The labels do not invent the distinction; these branches define it. We will ask the synthesizer specifically for the character branch.

Delivery: Initially emphasize Optional::None and the first cursor. Click to emphasize Optional::Some and reveal the second cursor. The code layout must not move.

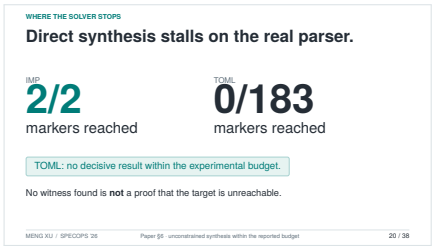
Precision: The cursor stays before the first closing character, not after it. Do not equate EOF marker with all malformed headers that eventually end at EOF. Both branches return an error document outcome.

---

20 / Direct synthesis stalls on the real parser.

10:05–10:30 · 25 seconds, including overlays

Review page 20; live PDF page(s) 28.



Paper §6 · unconstrained synthesis within the reported budget

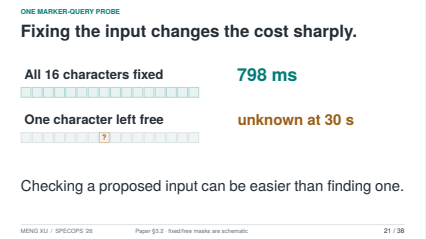
The semantics gives us precise questions, but that does not mean they are easy to solve. Z3 directly produces both IMP witnesses. For TOML, none of the 183 unconstrained queries yields a decisive result within the budget. That means neither a generated witness nor a proof of unreachability. The limitation is the whole-program encoding of this recursive parser on symbolic input.

Delivery: Pause on "no decisive result." Do not label the 183 targets unsatisfiable.

21 / Fixing the input changes the cost sharply.

10:30–10:55 · 25 seconds, including overlays

Review page 21; live PDF page(s) 29.



Paper §3.2 · fixed/free masks are schematic

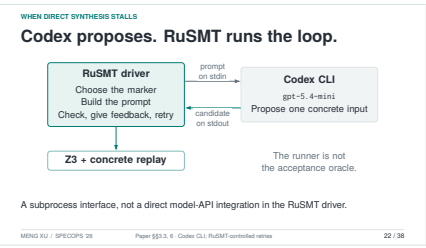
One probe makes the difference concrete. For one marker query, fixing all sixteen characters of a document gives a result in 798 milliseconds. Leave just one character symbolic, and the query returns unknown at thirty seconds. This is one reported probe, not an average. It motivates obtaining a complete candidate by another method while keeping the original target query for checking.

Delivery: Keep the scope explicit: one reported query probe, not a universal solver law.

22 / Codex proposes. RuSMT runs the loop.

10:55–11:30 · 35 seconds, including overlays

Review page 22; live PDF page(s) 30.



Paper §§3.3, 6 · Codex CLI;
RuSMT-controlled retries

The paper uses gpt-5.4-mini through the Codex command-line runner. The important distinction is who controls the experiment. RuSMT selects the target, assembles a prompt, sends it to the proposer on standard input, and reads a candidate from standard output. RuSMT then invokes Z3 and the executable semantics, constructs the feedback, and decides whether another round is needed. So a coding-agent runner is being used as a candidate-producing subprocess, not as the authority that grades its own test. The target still comes from the executable specification.

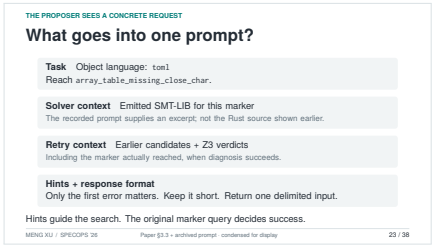
Delivery: Trace prompt to Codex and candidate back to RuSMT; then point to Z3 and replay. Name the model once.

Precision: The paper establishes Codex CLI as the runner. Do not infer that no model-service API is used underneath, or that the runner makes no internal tool calls. The recorded experiment captures final responses, not a complete agent tool trace.

23 / What goes into one prompt?

11:30–12:10 · 40 seconds, including overlays

Review page 23; live PDF page(s) 31.



Paper §3.3 + archived prompt · condensed for display

Here is the prompt structure, condensed from the recorded exchange. It names TOML and the precise error marker, supplies emitted SMT-LIB context, and includes earlier attempts with their solver feedback. It also gives concrete search advice: only the first error matters, so everything before the intended violation must be valid; shorter inputs reduce opportunities for an earlier error. Finally it specifies how to return the candidate. The archived prompt supplies an SMT-LIB excerpt, not the Rust code that we showed the audience. The model gets semantic context and feedback, but it is not given a guarantee that its interpretation of the marker is correct.

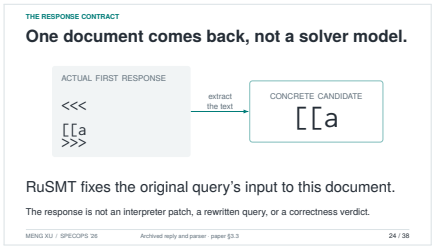
Delivery: Walk top to bottom. Keep the SMT-LIB body folded rather than teaching its syntax. Point out that first-error masking is already addressed in the prompt.

Precision: Prompt wording and excerpt formatting are illustrative archive details. The accepted paper supplies the experimental claims. Do not claim a prompt-length or source-visibility ablation, or attribute the misses to truncation.

24 / One document comes back, not a solver model.

12:10–12:35 · 25 seconds, including overlays

Review page 24; live PDF page(s) 32.



Archived reply and parser · paper §3.3

The requested output is not JSON, a program patch, or a proposed satisfying assignment for every variable. It is one concrete TOML document inside these delimiters. This is the actual first response. RuSMT extracts the enclosed text and encodes it as the entry function's input. Only then does Z3 determine whether that input satisfies the existing marker query. This is why I call the language model's output a candidate input, reserving solver model for Z3's satisfying assignment.

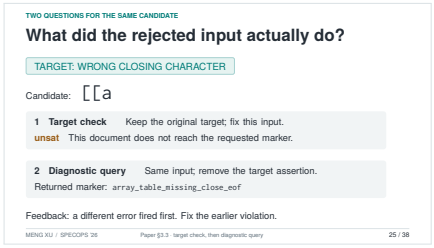
Delivery: Point to delimiters, then to the extracted input. Do not read every bracket aloud.

Precision: The archive parser accepts the <<<...>>> block, removes one framing newline at each end, and decodes only the special \u{...} transport escape. Other escapes remain literal. These serialization details are in the source package, not a new paper claim.

25 / What did the rejected input actually do?

12:35–13:10 · 35 seconds, including overlays

Review page 25; live PDF page(s) 33-34.



Paper §3.3 · target check, then diagnostic query

The first query keeps the target assertion and fixes the input to double bracket a. Z3 says unsat: this particular document does not reach our requested marker. RuSMT then runs a different diagnostic query. The input stays fixed, but the target assertion is removed so Z3 can report the outcome that actually occurs. That is the EOF marker, exactly the None branch we saw in the parser. RuSMT packages that observed marker into feedback for the next prompt. It is not asking the language model to invent an explanation of the rejection.

Delivery: Click once to reveal the diagnostic query and feedback. Tie the observed EOF marker to the earlier code trace.

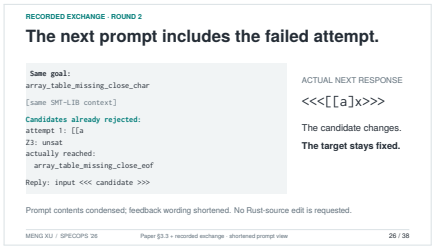
Precision: Pinned unsat rejects a candidate, not the marker. The diagnostic is another solver query, not an unsat core, a complete trace, or a proof of why the model reasoned incorrectly.

---

26 / The next prompt includes the failed attempt.

13:10–13:40 · 30 seconds, including overlays

Review page 26; live PDF page(s) 35-36.



Paper §3.3 + recorded exchange · shortened prompt view

The next attempt is not a new generic request to generate invalid TOML. RuSMT rebuilds the prompt with the same target and solver context, plus the failed candidate and the marker actually reached. The history is now explicit. The next response is double bracket a closing bracket x, returned through exactly the same text channel. The model changed its proposed document; it did not change the condition that counts as success. The full recorded prompts are in the source package, with this slide showing only the relevant fields.

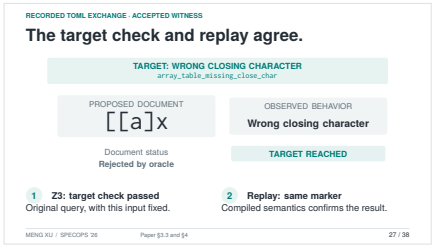
Delivery: Keep the target and context on screen. Reveal the next response after describing the added history.

Precision: This is a condensed reconstruction of the recorded prompt, not a verbatim screenshot. The returned candidate text is exact. A fresh proposer invocation receives the supplied history; do not describe this as training or cross-marker learning.

27 / The target check and replay agree.

13:40–14:05 · 25 seconds, including overlays

Review page 27; live PDF page(s) 37-38.



The candidate now passes the original marker query with the input fixed. It also reaches the same marker when run through the compiled semantics. Notice the separate meanings of success: the TOML document is rejected by the oracle, while the test succeeds at reaching the selected error. The model supplied text. The solver and replay determine whether it answers the testing request.

Paper §3.3 and §4

Delivery: Reveal target-check pass first, then concrete replay. Do not use bare accepted/rejected labels.

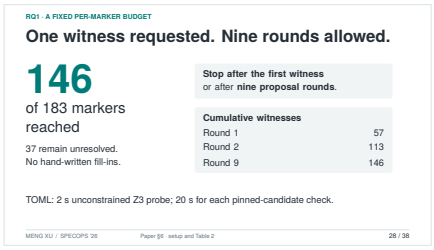
Precision: Replay checks this witness against the executable semantics; it does not verify the prose-to-code formalization.

---

28 / One witness requested. Nine rounds allowed.

14:05–14:30 · 25 seconds, including overlays

Review page 28; live PDF page(s) 39.



This is a budgeted search, not an instruction to keep working indefinitely. Each marker gets up to nine proposal rounds, with one witness requested. The reported run reaches 146 of the 183 markers. Fifty-seven arrive in round one and 113 cumulatively by round two; the later rounds bring the total to 146. The 37 remaining cases were not filled with hand-written tests. They are unresolved requests under this run and budget, not proofs that the branches are dead.

Paper §6 · setup and Table 2

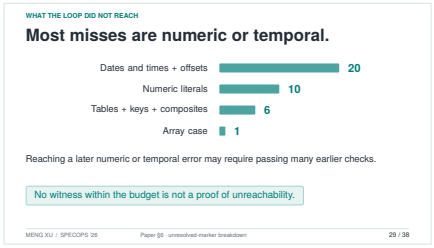
Delivery: Read 146 out of 183, then the stopping rule. Mention the early-versus-late progression without reading a full nine-row table.

Precision: Use the accepted paper: 2 s unconstrained probe; 20 s per pinned candidate; nine rounds; one witness. The paper reports no acceptance-check timeouts. Do not explain the 37 misses by claiming their pinned checks timed out. A rerun can differ.

29 / Most misses are numeric or temporal.

14:30–15:05 · 35 seconds, including overlays

Review page 29; live PDF page(s) 40.



Paper §6 · unresolved-marker breakdown

The misses are structured. Twenty concern dates, times, and numeric offsets; ten concern numeric literals, mostly float overflow and malformed radix forms; six concern table or key redefinition and unterminated composites; one is an array case. The paper links this pattern to earlier errors masking the target and to long chains of numeric or temporal subparsers. Naming the desired late error is easy; constructing a document that passes everything before it can be harder. This is evidence about where this loop struggled, not an experiment showing which larger model or prompt would fix it.

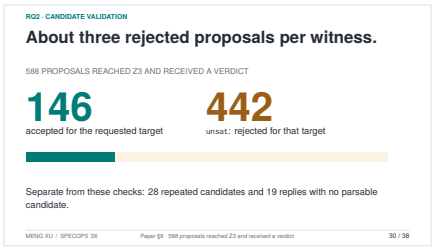
Delivery: Group the 20 and 10 first; do not read every subcategory. End on the unresolved-versus-unreachable distinction.

Precision: The paper calls this pattern consistent with parser structure. It does not causally isolate prompt length, reasoning effort, model choice, or the nine-round cutoff. Some misses are not deep numeric paths; retain all four reported groups.

30 / About three rejected proposals per witness.

15:05–15:30 · 25 seconds, including overlays

Review page 30; live PDF page(s) 41.



Paper §6 · 588 proposals reached Z3 and received a verdict

The loop received 588 proposals that reached Z3 and got a verdict: 146 passed and 442 were unsatisfiable for the requested target, about three rejections per accepted witness. Those 442 are not malformed model-output messages. They are extracted candidate inputs that fail the semantic target check. Separately, the paper reports 28 repeated candidates and 19 occasions with no parsable candidate response. That distinction matters because the main failure is not merely getting the model to follow a text format.

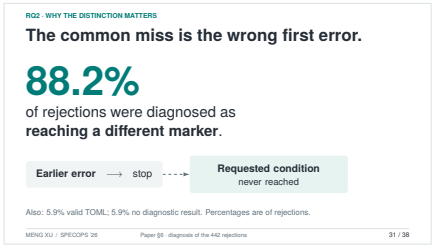
Delivery: Read the 146 versus 442 comparison, then distinguish semantic rejection from the two transport/history categories.

Precision: The 19 failures concern obtaining a parsable candidate response, not the expected invalidity of a TOML test document. Use the paper counts; do not recompute or substitute archive accounting.

31 / The common miss is the wrong first error.

15:30–15:55 · 25 seconds, including overlays

Review page 31; live PDF page(s) 42.



Paper §6 · diagnosis of the 442 rejections

Of the rejections, 88.2 percent were diagnosed as reaching a different marker. Another 5.9 percent were valid TOML, and for 5.9 percent the diagnostic returned nothing. Usually an earlier violation stops parsing before the requested condition, just as double bracket a reaches EOF instead of the character marker. This happens even though the prompt warns that only the first error matters. The checker is therefore enforcing a distinction that a plausible-looking candidate often misses.

Delivery: Connect the 88.2 percent result back to both the prompt hint and the concrete bracket exchange.

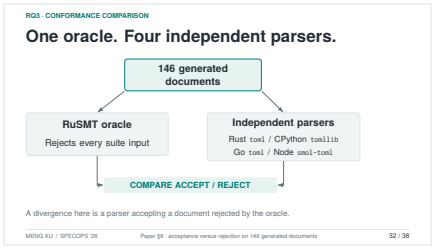
Precision: All percentages are of the 442 rejections, not all proposals or all markers. The diagnostic identifies an observed outcome, not the model's internal reasoning. No prompt or model ablation is reported.

---

32 / One oracle. Four independent parsers.

15:55–16:20 · 25 seconds, including overlays

Review page 32; live PDF page(s) 43.



Paper §6 · acceptance versus rejection on 146 generated documents

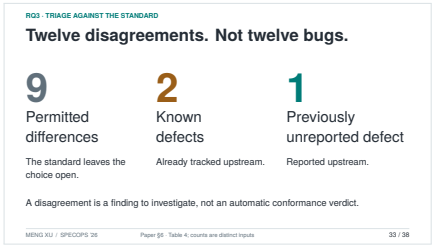
Return to the testing setup from the beginning. Now we have 146 generated documents. We run them through our oracle and four independent TOML parsers. Every document in this suite is rejected by the oracle, so a divergence is a production parser accepting it. We compare acceptance and rejection, not matching marker names or diagnostic wording. This is where the oracle supplies reference behavior for testing the implementations.

Delivery: Trace the two outgoing paths from the suite. Keep diagnosis and conformance comparison distinct.

33 / Twelve disagreements. Not twelve bugs.

16:20–16:45 · 25 seconds, including overlays

Review page 33; live PDF page(s) 44.



Paper §6 · Table 4; counts are distinct inputs

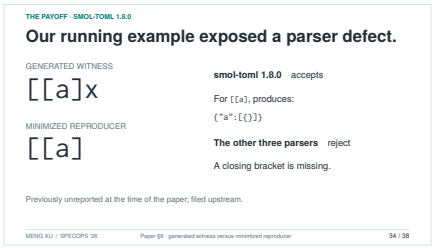
On twelve distinct inputs, the parsers and oracle do not all agree. Triage separates nine permitted differences, two already tracked defects, and one previously unreported defect. Twelve disagreements do not mean twelve bugs. A reference can make a representation choice that another implementation is allowed to make differently. The comparison identifies behavior to investigate; the standard tells us whether the difference violates a requirement.

Delivery: Count distinct inputs, not the sum of the per-parser divergence counts.

34 / Our running example exposed a parser defect.

16:45–17:30 · 45 seconds, including overlays

Review page 34; live PDF page(s) 45.



Paper §6 · generated witness versus minimized reproducer

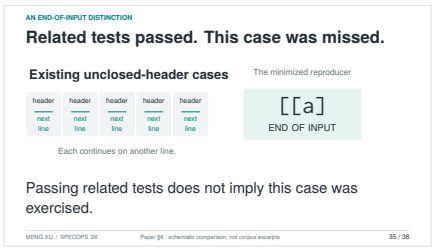
The previously unreported defect comes from our running example. The generated witness is double opening bracket a closing bracket x. The defect minimizes to the shorter header below, with only one closing bracket. smol-toml 1.8.0 accepts that malformed header and produces an array containing an empty table. The other three parsers reject it. The grammar requires two closing brackets, so this is not merely a diagnostic difference or a permitted numeric representation choice. The paper reports the defect upstream and contributes missing cases to the corpus. Keep the two strings distinct: synthesis produced the first; the second is the minimized reproducer.

Delivery: Return explicitly to the earlier proposal. Describe the version tested in the paper, not the current release.

35 / Related tests passed. This case was missed.

17:30–18:00 · 30 seconds, including overlays

Review page 35; live PDF page(s) 46.



Paper §6 · schematic comparison, not corpus excerpts

The subtle part is that the parser already passed all five existing toml-test cases for unclosed table headers. Each of those cases places another line after the malformed header. They do not exercise this end-of-input case. The left-hand documents are schematics, not excerpts of the actual corpus. This is why a targeted specification condition can still yield a useful test even when related examples already exist and pass.

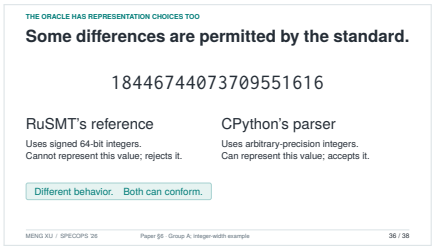
Delivery: Contrast continuation with end of input; do not claim the corpus had no malformed-header tests.

---

36 / Some differences are permitted by the standard.

18:00–18:35 · 35 seconds, including overlays

Review page 36; live PDF page(s) 47.



Paper §6 · Group A; integer-width example

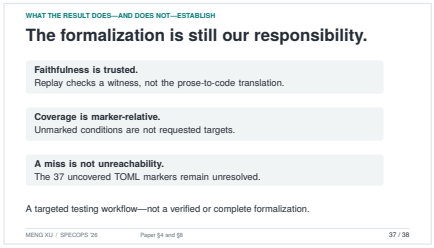
The nine permitted differences are important too. Consider this integer. Our reference uses signed 64-bit integers and cannot represent it. CPython can represent it exactly with arbitrary-precision integers. The TOML specification permits that implementation choice, so both behaviors can conform. A marker can identify a real branch of our executable model without identifying a mandatory distinction in the standard. The generated test can be right about the model, while the disagreement is not an implementation defect.

Delivery: Do not read all the digits. Say "an integer beyond the reference's range."

37 / The formalization is still our responsibility.

18:35–19:20 · 45 seconds, including overlays

Review page 37; live PDF page(s) 48.



Paper §4 and §8

There are three limits to keep separate. First, the executable specification is manually written and trusted to capture the standard; replay does not prove faithfulness to prose. Second, coverage is relative to the selected markers. An unmarked condition is not a requested target, and a representation choice is not automatically a standard requirement. Third, an unresolved marker is not a proof that no witness exists. These are boundaries of the result, not changes to the question we ask. The framework constructs and checks witnesses for selected conditions; it does not eliminate the work of writing the semantics or establish a complete language conformance proof.

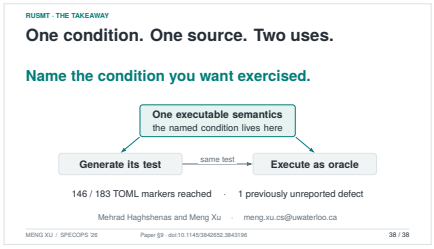
Delivery: Give each boundary a short explanation; do not read a disclaimer list at high speed.

---

38 / One condition. One source. Two uses.

19:20–20:00 · 40 seconds, including overlays

Review page 38; live PDF page(s) 49.



Paper §9 · doi:10.1145/3842652.3843196

We started by wanting to test language implementations, with an oracle and more explicit targets in the language semantics. The request is not just an invalid input, but an input that reaches this particular error condition. In RuSMT, the author writes an annotated executable interpreter, represents the error, and names its branch. The same source then gives us a synthesis query and an executable oracle. IMP makes that connection visible in code. TOML shows the synthesis boundary, the value of target checking, and a previously unreported parser defect. Name the condition you want exercised, and derive its test and its oracle from one executable semantics.

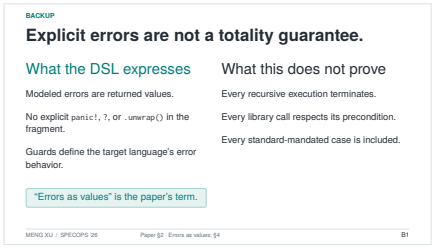
Delivery: End on this frame. No dedicated Q&A interval is reserved in the 20-minute plan.

Backup frames: outside the 20-minute talk

B1 / Explicit errors are not a totality guarantee.

Outside the timed talk

Review page 39; live PDF page(s) 50.



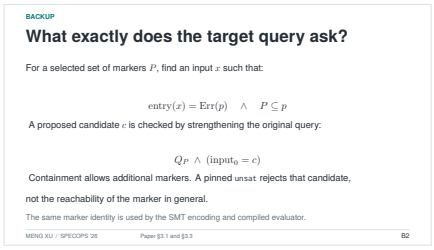
The supported claim is explicit error semantics. The paper explicitly notes that a library primitive can panic when used outside its documented precondition. The DSL also supports recursion and does not claim a termination proof for every program. Therefore this is not a totality or panic-freedom theorem. The author has a disciplined way to model errors and must guard primitive calls appropriately.

Paper §2 · Errors as values; §4

B2 / What exactly does the target query ask?

Outside the timed talk

Review page 40; live PDF page(s) 51.



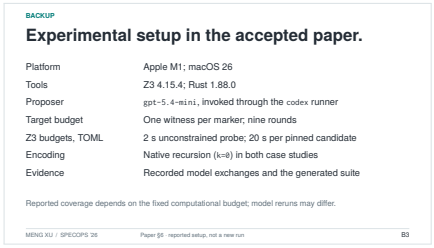
The query asserts that the returned path contains the requested marker set. Containment, rather than exact equality, allows an execution to reach additional markers. A candidate fixes the input in the original query. A satisfiable strengthened query still satisfies the original obligation. An unsatisfiable pinned query only rejects that input for that target; it does not show the target is unreachable.

Paper §3.1 and §3.3

B3 / Experimental setup in the accepted paper.

Outside the timed talk

Review page 41; live PDF page(s) 52.



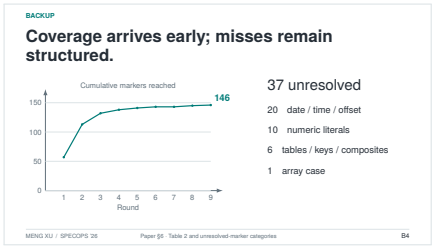
Paper §6 · reported setup, not a new run

These are the settings reported in the accepted paper. The round limit and budgets were fixed in advance. The model is nondeterministic, so the experimental artifact is the recorded run and the resulting suite, not a claim that every rerun will return the same documents. The Z3 timeout setting is not interchangeable with a wall-clock measurement.

B4 / Coverage arrives early; misses remain structured.

Outside the timed talk

Review page 42; live PDF page(s) 53.



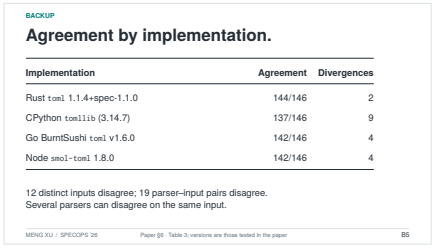
Paper §6 · Table 2 and unresolved-marker categories

The first two rounds produce 113 of the 146 witnesses. Later rounds add diminishing numbers. Misses are concentrated in date and time processing, numeric literals, and more composite or state-dependent conditions. The paper interprets this pattern as consistent with earlier errors masking the intended target and with long chains of numeric or temporal parsing. These are unresolved targets, not proven dead branches.

B5 / Agreement by implementation.

Outside the timed talk

Review page 43; live PDF page(s) 54.



BACKUP

Agreement by implementation.

Implementation	Agreement	Divergences
Rust `toml` 1.1.4+spec-1.1.0	144/146	2
CPython `tomllib` (3.14.7)	137/146	9
Go BurntSushi `toml` v1.6.0	142/146	4
Node `smol-toml` 1.8.0	142/146	4

12 distinct inputs disagree; 19 parser–input pairs disagree.
Several parsers can disagree on the same input.

MENG XU / SPECOPS '26 Paper §6 · Table 3; versions are those tested in the paper B5

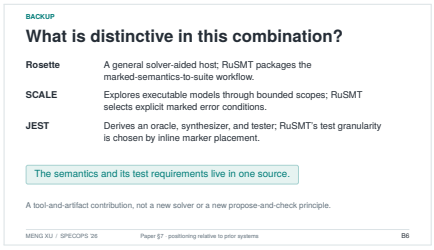
Paper §6 · Table 3; versions are those tested in the paper

The four divergence counts sum to nineteen parser–input pairs, not nineteen distinct documents. The paper's triage accounts for twelve distinct inputs. All suite documents are rejected by the oracle, so a disagreement in this table is a production implementation accepting a generated document. No successful parsed-value equivalence is evaluated in this table.

B6 / What is distinctive in this combination?

Outside the timed talk

Review page 44; live PDF page(s) 55.



BACKUP

What is distinctive in this combination?

Rosette	A general solver-aided host; RuSMT packages the marked-semantics-to-suite workflow.
SCALE	Explores executable models through bounded scopes; RuSMT selects explicit marked error conditions.
JEST	Derives an oracle, synthesizer, and tester; RuSMT's test granularity is chosen by inline marker placement.

The semantics and its test requirements live in one source.

A tool-and-artifact contribution, not a new solver or a new propose-and-check principle.

MENG XU / SPECOPS '26 Paper §7 · positioning relative to prior systems B6

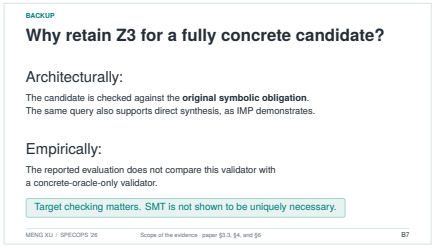
Paper §7 · positioning relative to prior systems

The paper does not claim to invent executable specifications, solver-backed DSLs, specification-derived testing, or propose-and-check synthesis. It identifies SCALE and JEST as particularly close systems. The distinctive packaging is a Rust-embedded executable semantics whose inline markers choose the test requirements, with a conformance oracle compiled from that same source. The case studies and the measured solver boundary are central parts of the tool-and-artifact contribution.

B7 / Why retain Z3 for a fully concrete candidate?

Outside the timed talk

Review page 45; live PDF page(s) 56.



Scope of the evidence · paper §3.3, §4, and §6

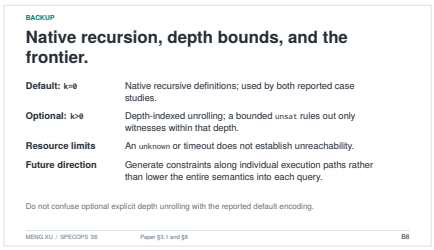
That is a fair distinction. Architecturally, retaining Z3 means every proposal is checked as a strengthening of the same query that can also perform direct synthesis. Concrete replay then checks the witness against the executable semantics. But the reported experiment does not compare this design to checking concrete candidates only by running the oracle. The rejection counts establish the importance of checking the intended target, not the unique necessity of an SMT solver for that task.

Delivery: This is an evidence-boundary answer, not a claim of a reported ablation result.

B8 / Native recursion, depth bounds, and the frontier.

Outside the timed talk

Review page 46; live PDF page(s) 57.



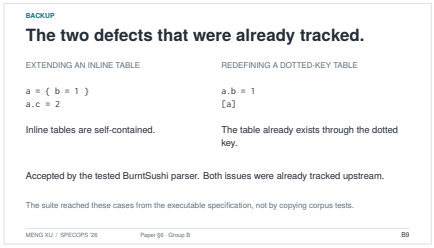
Paper §3.1 and §8

Both case studies use the native recursive encoding. The tool also offers explicit bounded unrolling, where unsatisfiability has only a bounded interpretation. Resource limits further restrict what can be established. The paper proposes path-wise constraint generation as a next direction, rather than requiring the solver to handle the entire recursive parser at once.

B9 / The two defects that were already tracked.

Outside the timed talk

Review page 47; live PDF page(s) 58.



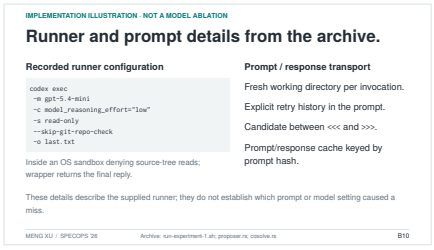
Paper §6 · Group B

The two known defects involve extending a self-contained inline table and redefining a table created by a dotted key. The specification prohibits both cases, and the tested Go parser accepts them. The paper checked that they were already documented through skipped upstream tests, so they were not filed as new findings. Reaching known defects from the executable specification is a check on the workflow, not a novelty claim about those bugs.

B10 / Runner and prompt details from the archive.

Outside the timed talk

Review page 48; live PDF page(s) 59.



Archive: run-experiment-1.sh; proposer.rs; cosolve.rs

These are the supporting implementation details from the archive, not extra evaluation claims. The experiment script selects low reasoning effort and launches Codex under a source-tree read restriction. The framework gives each invocation a fresh temporary working directory and sends its explicit per-marker history in the prompt. The wrapper returns the final reply, which the driver parses between delimiters. It records prompt-response pairs by a content-derived key. This establishes the transport and recorded configuration; it does not show that all tools were disabled or that low reasoning effort caused any particular failure.

Delivery: Use only for a question about the exact runner, prompt formatting, or reproducibility.

Precision: The script wraps this command in sandbox-exec and cats the last-message file. This is not a standalone reproduction command. Source restrictions do not prove zero tool use. The archive prompt text is truncated; the paper reports results but no context-length or reasoning-effort ablation.